

방법 1: 작동 원리 상세 설명서

로컬 DB + 시스템 프롬프트 방식의 근본 원리 이해

작성일: 2026-02-24

1. 근본적인 질문: 왜 이 방식인가?

1.1 문제 상황

사용자가 "다음 곡 뭐야?"라고 물었을 때, AI가 정확히 답하려면 무엇이 필요한가?

- 오늘 공연의 세트리스트가 뭔지
- 지금 몇 번째 곡을 하고 있는지
- 따라서 다음 곡이 뭔지

그런데... Gemini/ChatGPT는 이걸 모릅니다!

1.2 왜 AI는 이걸 모르는가?

Gemini/ChatGPT가 아는 것 (학습 데이터):

- BTS가 누구인지, 멤버들 이름, 유명한 노래들, K-POP 문화

Gemini/ChatGPT가 모르는 것:

- 오늘 2026년 6월 15일 서울 콘서트 세트리스트
- 지금 이 순간 몇 번째 곡을 하고 있는지
- 이 특정 콘서트의 응원법, 최신 팬덤 용어

이유: AI는 학습 시점까지의 데이터만 알고, 실시간 정보를 알 수 없음

1.3 해결책: 우리가 알려주면 된다!

AI가 모르는 정보를 우리가 매번 알려주면 됩니다.

예: "안녕 Gemini! 오늘 세트리스트는 1.Dynamite 2.좋아좋아 3.봄날이고, 지금 1번 곡 하고 있어. 팬이 질문할 거야, 대답해줘!"

팬: "다음 곡 뭐야?" → AI: "다음 곡은 좋아좋아야!" (정확한 답변 가능)

2. Gemini API가 필요한 이유

2.1 규칙 기반 vs AI 기반 비교

항목	규칙 기반 (AI 없음)	AI 기반 (Gemini)
다양한 표현	"다음에 뭐해?" 인식 못함	모든 표현 이해
오타 처리	"잔 생일?" 인식 못함	오타 허용
맥락 이해	"그 다음은?" 처리 불가	이전 대화 참고
유지보수	수백 개 if문 필요	프롬프트만 수정
응답 품질	"다음 곡: B" (딱딱)	"다음 곡은 B야!" (자연스러움)

2.2 Gemini가 하는 역할

Gemini가 하는 것:

- 자연어 이해: 다양한 표현을 같은 의도로 파악

- 정보 추출 및 조합: 주어진 정보에서 답 추론
- 자연스러운 문장 생성: 친근한 응답 생성

Gemini가 하지 않는 것:

- 셋리스트 정보 제공 (우리가 알려줌)
- 현재 곡 번호 추적 (앱이 관리)
- LED 색상 직접 제어 (앱이 처리)

3. 데이터셋: 언제, 어떻게 만드는가?

3.1 데이터의 3가지 계층

계층	내용	생성 시점	갱신 주기
Layer 1: 정적	멤버 프로필, 팬덤 용어	앱 개발 시	거의 없음 (연 1~2회)
Layer 2: 반정적	셋리스트, 응원법, 콘서트 정보	콘서트 전	투어 시작 전
Layer 3: 동적	현재 곡 번호 (인덱스)	실시간	곡 바뀔 때마다

3.2 데이터 생성 프로세스

[앱 개발 단계]

- 아티스트 기본 정보 수집 (공식 프로필, 팬덤 위키)
- JSON 파일로 정리 후 앱에 내장 (assets 폴더)

[콘서트 전]

- 셋리스트 확정 후 데이터 업데이트
- 앱 업데이트 배포 또는 서버에서 다운로드

[콘서트 중]

- 사용자가 이전/다음 버튼으로 현재 곡 직접 설정

4. Gemini가 데이터를 참고하는 방법

4.1 핵심 메커니즘: 시스템 프롬프트

★ 가장 중요한 개념 ★

Gemini는 데이터베이스에 접근하지 않습니다! 대신, 매 API 호출 시 텍스트로 정보를 전달받습니다.

작동 방식:

- STEP 1: 앱이 JSON에서 데이터 로드 (앱 시작 시)
- STEP 2: 사용자 질문 시 프롬프트 생성 (JSON 데이터 + 현재 상태를 텍스트로)
- STEP 3: Gemini API 호출 (프롬프트 + 질문 전송)
- STEP 4: 응답 수신 (Gemini는 프롬프트 정보를 읽고 답변)

4.2 시각적 비유: 시험지 + 컨닝 페이퍼

학생(Gemini)은 한국어는 잘하지만, 오늘 시험 범위(콘서트 정보)를 모릅니다. 그래서 우리가 컨닝 페이퍼(시스템 프롬프트)를 시험지(질문)와 함께 매번 건네줍니다. 학생은 컨닝 페이퍼를 보고 답을 작성합니다.

4.3 매 요청마다 정보를 보내는 이유

Q: 왜 매번 같은 정보를 보내야 하나요?

A: Gemini API는 상태 없음(Stateless)입니다.

API 호출이 끝나면 Gemini는 모든 것을 잊습니다. 다음 질문에서 맥락을 알려면 매번 필요한 정보를 함께 전송해야 합니다. 이것이 토큰 비용이 발생하는 이유입니다.

5. 일반 프롬프트 vs 시스템 프롬프트

5.1 어차피 Gemini면 아무 질문이나 가능한 거 아닌가?

시나리오 A: 시스템 프롬프트 없이

사용자: "다음 곡 뭐야?" → Gemini: "무슨 다음 곡을 말씀하시는 건가요? 구체적으로 알려주시면..."

시나리오 B: 시스템 프롬프트 포함

[시스템 프롬프트] "셋리스트: 1.Dynamite 2.좋아좋아, 현재 1번"

사용자: "다음 곡 뭐야?" → Gemini: "다음 곡은 좋아좋아야!"

5.2 시스템 프롬프트의 3가지 역할

역할	설명	예시
컨텍스트 제공	AI가 답변에 필요한 사실 정보	현재 곡: 1번, 다음 곡: 2번
페르소나 설정	AI의 말투, 성격, 행동 방식	반말로 대화, 이모지 사용
규칙 설정	응답의 길이, 형식, 제약 조건	1-2문장, LED 색상 추천 포함

5.3 일반적인 질문도 가능한가?

가능합니다! Gemini의 기본 지식으로 답변하되, 콘서트 맥락을 유지합니다.

예: "서울 날씨 어때?" → "실시간 날씨는 확인 못 해! 근데 지금은 Dynamite 응원에 집중!"

6. 대안: LLaMA 등 로컬 LLM 내장

6.1 클라우드 LLM vs 로컬 LLM

항목	클라우드 (Gemini)	로컬 (LLaMA)
인터넷	필수	불필요
응답 속도	1~3초	3~10초
응답 품질	최고 (대형 모델)	보통~양호 (경량 모델)
앱 용량	~50MB	2~5GB
API 비용	월 \$5~20	\$0
배터리 소모	낮음	높음
발열	거의 없음	상당함
지원 기기	모든 스마트폰	고성능 폰만 (RAM 6GB+)
한국어 품질	매우 좋음	모델에 따라 다름
개발 난이도	쉬움	어려움

6.2 로컬 LLM의 현실적 문제점

- 모델 용량: 양자화된 7B 모델도 4~5GB (앱 설치 용량 5GB 이상)
- 초기 로딩: 모델 로딩 10~30초 (콘서트장에서 급히 켜야 하는데...)
- 한국어 품질: 대부분 영어 중심, 한국어 응답 어색할 수 있음
- 발열/배터리: 연속 질문 시 발열, 배터리 급격히 소모

6.3 핵심 포인트: 시스템 프롬프트 방식은 동일

클라우드 LLM이든 로컬 LLM이든, 시스템 프롬프트로 데이터를 전달하는 방식은 동일합니다!

JSON 데이터 → 시스템 프롬프트 생성 → (Gemini / LLaMA / Claude 등) → 어떤 LLM을 쓰든 같은 방식

7. 방식별 비교 분석

방식	장점	단점	적합한 경우
A: 규칙 기반 (AI 없음)	가장 빠름, 비용 없음 오프라인 가능	자연어 이해 불가 유지보수 어려움	매우 단순한 앱 (버튼식 UI)
B: 클라우드 LLM (Gemini) ★권장	자연어 이해, 고품질 개발 쉬움	인터넷 필요 API 비용	대부분의 AI 앱 MVP 개발
C: 로컬 LLM (LLaMA)	오프라인 가능 API 비용 없음	느림, 앱 용량 큼 품질 제한	오프라인 필수 개인정보 민감
D: 하이브리드 (로컬+클라우드)	비용 최적화 오프라인 부분 지원	개발 복잡 일관성 관리 어려움	고도화된 상용 앱

8. 결론 및 권장사항

8.1 MVP 단계 권장 구성

★ 권장: 클라우드 LLM (Gemini) + 시스템 프롬프트

이유:

- 개발 가장 빠름 (5일 내 가능)
- 응답 품질 최고 (한국어 자연스러움)
- 비용 저렴 (월 \$5~10 수준)
- 콘서트장 인터넷 가능 (문제 없음)
- 모든 스마트폰 지원

8.2 전체 작동 원리 요약

단계	내용
1. 데이터 준비	JSON 파일에 콘서트 정보 저장 (멤버, 셋리스트, 팬덤 용어)
2. 앱 실행	JSON 로드 → 메모리에 저장, 현재 곡 인덱스 = 0
3. 사용자 질문	"다음 곡 뭐야?" (음성 또는 텍스트)
4. 프롬프트 생성	JSON 데이터 + 현재 상태 → 시스템 프롬프트 생성
5. AI API 호출	시스템 프롬프트 + 사용자 질문 → Gemini 전송
6. 응답 처리	텍스트 응답 → TTS 출력, 색상 추출 → LED 제어

핵심 원리

AI는 데이터베이스에 접근하지 않는다. 우리가 매번 필요한 정보를 텍스트로 알려주고, AI는 그 텍스트를 읽고 자연스럽게 답변을 생성한다.

8.3 API 비용 참고

항목	수치
Gemini 2.0 Flash 입력	\$0.075 / 100만 토큰
Gemini 2.0 Flash 출력	\$0.30 / 100만 토큰
1회 질문 비용	약 \$0.0001 (0.1원)
콘서트 중 50회 질문	약 \$0.005 (5원)
월 1,000명 x 50회	약 \$5